

Data Mining (EECS 4412)

Sequential Pattern Mining

Aijun An

Department of Electrical Engineering and Computer Science
York University

Outline

- ▶ Basic concepts of sequential pattern mining
- ▶ A Simplified Version of GSP Algorithm
- ▶ PrefixSpan

An Example Sequence Database

Customer Id	Transaction Time	Items Bought
1	June 25 '93	30
1	June 30 '93	90
2	June 10 '93	10, 20
2	June 15 '93	30
2	June 20 '93	40, 60, 70
3	June 25 '93	30, 50, 70
4	June 25 '93	30
4	June 30 '93	40, 70
4	July 25 '93	90
5	June 12 '93	90

Figure 1: Database Sorted by Customer Id and Transaction Time

Customer Id	Customer Sequence
1	{ (30) (90) }
2	{ (10 20) (30) (40 60 70) }
3	{ (30 50 70) }
4	{ (30) (40 70) (90) }
5	{ (90) }

Figure 2: Customer-Sequence Version of the Database

- ▶ A *sequence database* consists of a set of sequences
- ▶ A *sequence* is an ordered list of itemsets
- ▶ *Itemset*: a set of items
- ▶ Items within an itemset are unordered.
- ▶ *Element*: an itemset in a sequence

Example of a Sequential Pattern

- ▶ Database of an online book store
 - ▶ Contains data sequences
 - ▶ Each sequence corresponds to a list of transactions by a given customer.
 - ▶ Each transaction contains the books selected by the customer in one order.
- ▶ A sequential pattern
 - ▶ 5% of customers bought “*Foundation*”, then “*Foundation and Empire*” and “*Ringworld*”, then “*Second Foundation*”.
- ▶ Usefulness
 - ▶ Making recommendations.
 - ▶ Knowing what to stock.

Mining Sequential Patterns

- ▶ Objective
 - ▶ Finding (interesting) frequent sequences from a sequence database.
- ▶ Applications
 - ▶ Customer shopping sequence analysis
 - ▶ Web log analysis
 - ▶ DNA or protein analysis
 - ▶ Stock market analysis
 - ▶ Medical domain
 - ▶ Predict onset of a disease from a sequence of symptoms, etc.
 - ▶ E.g., <fever → cough → fatigue → diagnosis of pneumonia>
 - ▶ Earthquake prediction
 - ▶ etc.

Basic Concepts

- ▶ A sequence $\langle a_1, a_2, \dots, a_n \rangle$ *is contained in* (or *is a subsequence of*) another sequence $\langle b_1, b_2, \dots, b_m \rangle$ if there exist integers $i_1 < i_2 < \dots < i_n$ such that $a_1 \subseteq b_{i_1}, a_2 \subseteq b_{i_2}, \dots, a_n \subseteq b_{i_n}$.
 - ▶ E.g., $\langle (3) (4\ 5) (8) \rangle$ is contained in $\langle (7) (3\ 8) (9) (4\ 5\ 6) (8) \rangle$
- ▶ *Support of a sequence* is defined as the fraction of total sequences in the database that contains this sequence.
 - ▶ Support of $\langle (30) (90) \rangle$?
 - ▶ Support of $\langle (20) (60\ 70) \rangle$?
 - ▶ Support of $\langle (30) (70) \rangle$?
- ▶ *Frequent sequences* (also called *sequential patterns*)
 - ▶ sequences that satisfy a minimum support (min_sup).

Customer Id	Customer Sequence
1	$\langle (30) (90) \rangle$
2	$\langle (10\ 20) (30) (20\ 60\ 70) \rangle$
3	$\langle (30\ 50\ 70) \rangle$
4	$\langle (30) (40\ 60\ 70) (90) \rangle$
5	$\langle (90) \rangle$

Sequential Pattern Mining

- ▶ What is sequential pattern mining
 - ▶ Given a sequence database, find the set of frequent sequences that satisfy a minimum support (min_sup).
- ▶ Algorithms
 - ▶ Initial Apriori-like algorithms (Agrawal and Srikant, 95)
 - ▶ AprioriAll, AprioriSome, and DynamicSome
 - ▶ GSP - an Apriori-like, influential mining method (Srikant and Agrawal, 96)
 - ▶ PrefixSpan (Pei, et al, 01)
 - ▶ SPADE (Zaki, 01)
 - ▶ Mining sequential patterns with constraints (Pei, et al, 02)
 - ▶ etc.

Apriori Property

- ▶ The **Apriori** property in sequential patterns:
Any nonempty subsequence of a frequent sequence must be frequent
- ▶ If a sequence is infrequent, then none of its super-sequences is frequent.
- ▶ i.e., if $\langle (3) (4\ 5) \rangle$ is infrequent, so are $\langle (3) (4\ 5) (8) \rangle$ and $\langle (3\ 6) (4\ 5) \rangle$

GSP

- ▶ GSP: Generalized Sequential Pattern Mining
- ▶ Proposed in
 - ▶ R. Srikant and R. Agrawal. Mining Sequential Patterns: Generalizations and Performance Improvements. In Proc. of EDBT'96.
 - ▶ Can be downloaded from the course web site
- ▶ GSP considers some time constraints and item taxonomy
 - ▶ More general than simply mining sequential patterns

A Simplified Version of GSP

- ▶ GSP without the generalizations
- ▶ Problem statement:
 - ▶ find all frequent sequences from a database of sequences given a *min_sup*
- ▶ **Length of a sequence** = number of items in the sequence
 - ▶ Length of $\langle (a)(b) \rangle$ is 2
 - ▶ Length of $\langle (a\ b) \rangle$ is 2
 - ▶ Length of $\langle (a\ b)\ (c) \rangle$ is 3
- ▶ A **length- k sequence** is also called *k-sequence*.

General Description of Simplified GSP

▶ Method

- ▶ Take sequences in form of $\langle x \rangle$ as length-1 candidates
- ▶ Scan database once, find L_1 , the set of length-1 sequential patterns
- ▶ Let $k=1$; while L_k is not empty do
 - ▶ **Form** C_{k+1} , the set of length- $(k+1)$ **candidates** from L_k ;
 - ▶ If C_{k+1} is not empty, **scan database** once, find L_{k+1} , the set of length- $(k+1)$ sequential patterns
 - ▶ Let $k=k+1$;

Finding Length-1 Sequential Patterns

- ▶ Initial candidates

- ▶ $\langle(a)\rangle, \langle(b)\rangle, \langle(c)\rangle, \langle(d)\rangle, \langle(e)\rangle, \langle(f)\rangle, \langle(g)\rangle, \langle(h)\rangle$

- ▶ Scan database once

- ▶ count support for candidates

$min_sup = 40\%$

$min_sup_count = 2$

Seq-id	Sequence
10	$\langle(bd)(c)(b)(ac)\rangle$
20	$\langle(bf)(ce)(b)(fg)\rangle$
30	$\langle(ah)(bf)(a)(b)(f)\rangle$
40	$\langle(be)(ce)(d)\rangle$
50	$\langle(a)(bd)(b)(c)(b)(ade)\rangle$

Cand	Sup
$\langle(a)\rangle$	3
$\langle(b)\rangle$	5
$\langle(c)\rangle$	4
$\langle(d)\rangle$	3
$\langle(e)\rangle$	3
$\langle(f)\rangle$	2
$\langle(g)\rangle$	1
$\langle(h)\rangle$	1

Length-2 Candidate Generation

- ▶ How to generate C_2 from L_1
 - ▶ Merge every pair of frequent length-1 sequences.
 - ▶ Merging two frequent length-1 sequences $\langle i_1 \rangle$ and $\langle i_2 \rangle$ will produce three candidate 2-sequences:
 $\langle i_1 i_2 \rangle$ $\langle i_2 i_1 \rangle$ and $\langle i_1 i_1 \rangle$
assuming that i_1 is different from i_2 .
 - ▶ Every frequent length-1 sequence $\langle i \rangle$ can join with itself to produce $\langle i i \rangle$.

Length-2 Candidates

(Cont'd from the example on Slide 13)

Length of a sequence = the number of items

51 length-2 Candidates

	<a>		<c>	<d>	<e>	<f>
<a>	<(a)(a)>	<(a)(b)>	<(a)(c)>	<(a)(d)>	<(a)(e)>	<(a)(f)>
	<(b)(a)>	<(b)(b)>	<(b)(c)>	<(b)(d)>	<(b)(e)>	<(b)(f)>
<c>	<(c)(a)>	<(c)(b)>	<(c)(c)>	<(c)(d)>	<(c)(e)>	<(c)(f)>
<d>	<(d)(a)>	<(d)(b)>	<(d)(c)>	<(d)(d)>	<(d)(e)>	<(d)(f)>
<e>	<(e)(a)>	<(e)(b)>	<(e)(c)>	<(e)(d)>	<(e)(e)>	<(e)(f)>
<f>	<(f)(a)>	<(f)(b)>	<(f)(c)>	<(f)(d)>	<(f)(e)>	<(f)(f)>

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Without Apriori property,

$$8 \times 8 + 8 \times 7 / 2 = 92$$

candidates

Apriori prunes

44.57% candidates

Finding Length-2 Sequential Patterns

- ▶ Scan database one more time, collect support count for each length-2 candidate
- ▶ There are 19 length-2 candidates which pass the minimum support threshold in our example DB
 - ▶ They are length-2 sequential patterns:
<(a)(a)>, <(a)(b)>
<(b)(a)>, <(b)(b)>, <(b)(c)>, <(b)(d)>, <(b)(e)>, <(b)(f)>
<(c)(a)>, <(c)(b)>, <(c)(d)>
<(d)(a)>, <(d)(b)>, <(d)(c)>
<(f)(b)>, <(f)(f)>
<(bd)>, <(bf)>, <(ce)>

Generating C_k from L_{k-1} ($k > 2$)

- ▶ Join step: generate C_k by joining L_{k-1} with L_{k-1}
 - ▶ All items in an itemset are ranked in an order.
 - ▶ A sequence s_1 joins with s_2 *if the subsequence obtained by dropping the first item of s_1 is the same as the subsequence obtained by dropping the last item of s_2 .*
 - ▶ The joined sequence is s_1 plus the last item of s_2 .
 - ▶ The added item becomes a separate element if it was a separate element in s_2 , or part of the last element of s_1 otherwise.
 - ▶ Examples:
 - ▶ Joining $\langle(1)(2\ 3)(4)\rangle$ and $\langle(2\ 3)(4)(5)\rangle$ produces $\langle(1)\ (2\ 3)(4)(5)\rangle$
 - ▶ Joining $\langle(1)(2\ 3)(4)\rangle$ and $\langle(2\ 3)(4\ 5)\rangle$ produces $\langle(1)(2\ 3)(4\ 5)\rangle$

Generating C_k from L_{k-1} ($k > 2$) (Cont'd)

- ▶ Prune step: delete candidates in C_k that have infrequent $(k-1)$ -subsequence
 - ▶ A $(k-1)$ -subsequence of sequence s is derived by dropping an item from s
 - ▶ Check each $(k-1)$ -subsequence against L_{k-1}
 - ▶ Example:
 - ▶ If $\langle (ab)(d) \rangle$, $\langle (b)(ad) \rangle$, $\langle (b)(de) \rangle$ are all length-3 frequent sequences, then
 - ▶ Join result: $\langle (ab)(de) \rangle$
 - ▶ Its length-3 subsequences:
 - ▶ $\langle (b)(de) \rangle$, $\langle (a)(de) \rangle$, $\langle (ab)(e) \rangle$, $\langle (ab)(d) \rangle$
 - ▶ $\langle (a)(de) \rangle$ and $\langle (ab)(e) \rangle$ are infrequent, $\langle (ab)(de) \rangle$ is pruned
 - ▶ As long as one of them is infrequent, can stop checking others.
 - ▶ No need to check $\langle (ab)(d) \rangle$ and $\langle (b)(de) \rangle$ (Why?)
 - ▶ C_4 becomes empty

Generating C_k from L_{k-1} ($k > 2$) (Cont'd)

▶ More Examples:

- ▶ If $\langle(a)(b)\rangle$, $\langle(a)(a)\rangle$ and $\langle(b)(a)\rangle$ are all length-2 sequential patterns, then length-3 candidates are:
 - ▶ Join result:
 - ▶ $\langle(a)(b)(a)\rangle$, $\langle(a)(a)(b)\rangle$, $\langle(a)(a)(a)\rangle$, $\langle(b)(a)(b)\rangle$, and $\langle(b)(a)(a)\rangle$.
 - ▶ After pruning:
 - ▶ $\langle(a)(b)(a)\rangle$, $\langle(a)(a)(b)\rangle$, $\langle(a)(a)(a)\rangle$, $\langle(b)(a)(a)\rangle$.
- ▶ If $\langle(bd)\rangle$, $\langle(b)(b)\rangle$ and $\langle(d)(b)\rangle$ are all length-2 sequential patterns, what are the length-3 candidates?
 - ▶ Join result:
 - ▶ $\langle(bd)b\rangle$, $\langle(b)(bd)\rangle$, $\langle(b)(b)(b)\rangle$, $\langle(d)(bd)\rangle$, $\langle(d)(b)(b)\rangle$
 - ▶ After pruning:
 - ▶ $\langle(bd)b\rangle$, $\langle(b)(b)(b)\rangle$, $\langle(d)(b)(b)\rangle$

Example Continued

- ▶ L_4 : Length-4 sequential patterns: $min_sup_count=2$

- ▶ $\langle (b)(c)(b)(a) \rangle$
- ▶ $\langle (bd)(b)(a) \rangle$
- ▶ $\langle (bd)(b)(c) \rangle$
- ▶ $\langle (bd)(c)(a) \rangle$
- ▶ $\langle (bd)(c)(b) \rangle$
- ▶ $\langle (bf)(b)(f) \rangle$
- ▶ $\langle (d)(c)(b)(a) \rangle$

Seq-id	Sequence
10	$\langle (bd)(c)(b)(ac) \rangle$
20	$\langle (bf)(ce)(b)(fg) \rangle$
30	$\langle (ah)(bf)(a)(b)(f) \rangle$
40	$\langle (be)(ce)(d) \rangle$
50	$\langle (a)(bd)(b)(c)(b)(ade) \rangle$

- ▶ C_5 : Length-5 candidates (after join and prune):
 - ▶ $\langle (bd)(c)(b)(a) \rangle$
- ▶ L_5 : Length-5 sequential pattern (found by scanning DB):
 - ▶ $\langle (bd)(c)(b)(a) \rangle$: 2

Summary of Simplified GSP

- ▶ Make the first pass over the sequence database D to find all the 1-element (length-1) frequent sequences
- ▶ Repeat until no new candidate or frequent sequences are found
 - ▶ **Candidate Generation:**
 - ▶ **Merge** joinable pairs of frequent sequences of length $(k-1)$ to generate candidate sequences that contain k items
 - ▶ **Prune** candidate k -sequences that contain infrequent $(k-1)$ -subsequences
 - ▶ **Support Counting and Candidate Elimination:**
 - ▶ Make a new pass over the sequence database D to find the support count for each candidate sequence
 - ▶ Eliminate candidate k -sequences whose actual support is less than min_sup